

Build Sequence

Eight phases, run one at a time. Each phase has an objective, an ordered task list, the code contracts it must satisfy, and an exit gate you can verify yourself before we move on. Nothing is implemented until you say “start Phase 0”.

PHASES

0 → 6, plus roadmap

EFFORT

~8 working days

RUNNING COST

₹0 – permanently

DATA SOURCE

Your CV only

SHIPPABLE AT

End of Phase 4

§ WORKING AGREEMENT

How each phase runs

One phase per session. I build it, then hand you the exit checklist. You verify against it — every criterion is something you can see or run yourself, not something you have to take my word for. Only once it passes do we open the next phase.

If a phase reveals something the plan got wrong, we amend the plan first and the code second. The plan stays the source of truth.

STANDING RULES – EVERY PHASE, NO EXCEPTIONS

- R1

No facts in components. No name, date, email, URL, number, or sentence about you appears inside `components/`. It comes from `content/` as props. Enforced by a pre-commit scan.
- R2

No raw colours or sizes. Every colour, space, radius, and font size is a design token. No hex codes, no arbitrary Tailwind values in class names. Enforced by lint.

- R3 No magic motion numbers.** Every duration, delay, stagger, spring, and particle count comes from `config/motion.ts`. A bare number in a transition prop fails lint.
-
- R4 One reader of the environment.** `process.env` is referenced in exactly one file, `config/env.ts`, and validated there. Enforced by lint.
-
- R5 Derived, never duplicated.** If a value can be computed from content, it is computed — counts, durations, metadata, suggestions, the search index. Nothing is typed in twice.
-
- R6 Typed strictly.** `strict` plus `noUncheckedIndexedAccess`. No `any`. Schemas infer the types, so data and types can never drift apart.
-
- R7 Nothing metered.** No dependency that can generate a bill may enter the project. If a library needs an account, it does not go in.
-
- R8 Accessible by default.** Keyboard reachable, visible focus, semantic landmarks, and a real static alternative under reduced motion — built in as we go, never retrofitted.
-

CODE CONVENTIONS

AREA	CONVENTION
Files	<code>PascalCase.tsx</code> for components, <code>camelCase.ts</code> for logic, <code>kebab-case</code> for routes.
Exports	Named exports everywhere except Next.js route files, which require default. Makes every import greppable.
Components	Server Components by default. <code>'use client'</code> only where interaction or canvas demands it, and pushed as deep in the tree as possible.
Props	Explicit interface per component, no prop spreading of unknown shapes. Content types come from the schema, never redeclared.
Logic	Pure functions in <code>lib/</code> , unit-testable without React. Components render; they do not compute.
Comments	Only to record a constraint the code cannot express. Not to narrate what the next line does.
Commits	Conventional commits, one phase task per commit, so the history reads as this plan.

A deployed, empty, correctly-wired shell

Nothing visible gets built in this phase. The point is that every rule above becomes *mechanically enforced* before there is any code to break it. Rules added after the fact never hold.

TASKS

- 0.1 Scaffold Next.js 15 — TypeScript, App Router, Tailwind v4, ESLint, `src/` directory. `npm create next-app@latest`
- 0.2 `git init`, initial commit, push to a GitHub repo.
- 0.3 Tighten `tsconfig`: `strict`, `noUncheckedIndexedAccess`, `noImplicitOverride`, path alias `@/*`.
- 0.4 Install runtime deps: `zod`, `motion`, `lucide-react`, `clsx`, `tailwind-merge`. Dev: `vitest`, `@playwright/test`, `prettier`, `husky`, `lint-staged`.
- 0.5 Create the full directory skeleton with placeholder barrel files, so structure exists before content does.
- 0.6 Write `config/env.ts` — the single validated reader of `process.env`.
- 0.7 ESLint rules that enforce R1–R4: ban `process.env` outside `config/`, ban hex literals and arbitrary values in `className`, ban numeric literals in `transition` props.
- 0.8 Write `scripts/check-literals.ts` — scans `components/` for prose, dates, URLs, and emails; exits non-zero on a hit.
- 0.9 Husky pre-commit hook: typecheck → lint → literal scan → tests.
- 0.10 Import the repo into Vercel; confirm the automatic preview-per-branch deploy works.

```
// The ONLY file in the project permitted to read process.env.
// Everything else imports the parsed, typed object.

import { z } from 'zod'

const schema = z.object({
  NEXT_PUBLIC_SITE_URL: z.string().url().default('http://localhost:3000'),
  NEXT_PUBLIC_ENABLE_BLOG: z.enum(['true', 'false']).default('false'),
  NEXT_PUBLIC_ENABLE_ANALYTICS: z.enum(['true', 'false']).default('false'),
})

const parsed = schema.safeParse(process.env)
if (!parsed.success) {
  // Fail loudly at build, never silently at runtime.
  throw new Error('Invalid environment:\n' + parsed.error.toString())
}

export const env = parsed.data

// Note: no secrets. There is no API key anywhere in this project.
// A fresh clone runs with zero configuration.
```

EXIT GATE — PHASE 0

- `npm run build` completes with no errors or warnings.
- A live Vercel URL serves a blank styled page.
- Adding `process.env.FOO` to any component **fails lint**.
- Committing a component containing `#1B4DE4` or the word “Faiz” **is blocked by the pre-commit hook**.
- A fresh git clone plus `npm install` plus `npm run dev` works with no `.env` file at all.

01 CONTENT
LAYER05 DAY
Every fact from your CV, typed and validated

After this phase, your entire professional identity exists as validated data. Nothing else in the codebase will ever need to know a fact about you again.

TASKS

- 1.1 Write `content/schema.ts` — Zod schemas for every content type.
- 1.2 Enter real data: `profile.ts`, `experience.ts` (both internships), `projects.ts` (all four), `skills.ts`, `achievements.ts`, `education.ts`.
- 1.3 Seed `content/vocabulary.ts` — synonyms and aliases the assistant will use in Phase 5. Grows over time; starts small.

- 1.4 Define `content/sections.ts` — the ordered section registry that drives nav, scroll-spy, page composition, and footer.

- 1.5 Write `content/index.ts` — parses everything on import, cross-validates references, throws with a readable message on bad data.

- 1.6 Write `content/derived.ts` — computed statistics. Nothing here is typed by hand.

- 1.7 Write `content/format.ts` — date and duration formatters. Dates are stored as ISO, never as display strings.

- 1.8 Write `content.test.ts` — validation passes, cross-references resolve, derived stats match expectations.

CONTRACT – CORE SCHEMAS

```
// content/schema.ts – the contract. Types are inferred from these,
// so the data and the types can never drift apart.

export const LinkSchema = z.object({
  label: z.string().min(1),
  href: z.string().url(),
  kind: z.enum(['repo', 'live', 'doc', 'social', 'email']),
})

export const ExperienceSchema = z.object({
  id: z.string().regex(/^[a-z0-9-]+$/),
  org: z.string().min(1),
  role: z.string().min(1),
  start: z.string().regex(/^\d{4}-\d{2}$/), // ISO year-month
  end: z.string().regex(/^\d{4}-\d{2}$/).nullable(), // null = present
  summary: z.string().min(1),
  highlights: z.array(z.string()).min(1),
  stack: z.array(z.string()).default([]),
})

export const ProjectSchema = z.object({
  id: z.string().regex(/^[a-z0-9-]+$/),
  name: z.string().min(1),
  year: z.number().int(),
  problem: z.string().min(1), // what needed solving
  approach: z.string().min(1), // what you built
  outcome: z.string().min(1), // what it achieved
  stack: z.array(z.string()).min(1),
  highlights: z.array(z.string()).default([]),
  links: z.array(LinkSchema).default([]),
  featured: z.boolean().default(true),
})

export const SkillClusterSchema = z.object({
  id: z.string(),
  title: z.string(),
  blurb: z.string(),
  items: z.array(z.string()).min(1),
  evidence: z.array(z.string()).default([]), // project ids – cross-checked below
  level: z.number().min(0).max(100).optional(), // only if you choose bars
})

export const SectionSchema = z.object({
  id: z.string(),
  navLabel: z.string(),
  order: z.number().int(),
  stage: z.enum(['attention', 'understanding', 'proof', 'interaction', 'connection']),
  enabled: z.boolean().default(true),
})
```

CONTRACT – CROSS-REFERENCE VALIDATION

```
// content/index.ts – catches broken references at BUILD time,
// not as a blank space on the live page.

const projectIds = new Set(projects.map(p => p.id))

skills.forEach(cluster => {
  cluster.evidence.forEach(id => {
    if (!projectIds.has(id)) {
      throw new Error(
        `Skill cluster "${cluster.id}" cites unknown project "${id}". ` +
        `Known projects: ${[...projectIds].join(', ')}`
      )
    }
  })
})
```

CONTRACT – DERIVED STATISTICS

```
// content/derived.ts – R5 in practice. Add a project, and the hero
// counters update themselves. They cannot drift out of sync.

export const stats = {
  projectCount: projects.length,
  technologyCount: new Set(projects.flatMap(p => p.stack)).size,
  placementCount: achievements.filter(a => a.kind === 'placement').length,
  experienceMonths: monthsSince(earliestStart(experience)),
}
```

ONE TRAP TO AVOID

`experienceMonths` depends on today's date. Computing it in a Client Component causes a hydration mismatch, because the server and the browser can evaluate it at different moments. It is computed once in a Server Component and passed down as a plain number. Same rule for anything time-dependent.

EXIT GATE – PHASE 1

- `npm test` passes; all content validates.
- Changing a skill cluster's evidence to a non-existent project id **fails the build with a message naming the bad id and listing valid ones**.
- Deleting a project makes `stats.projectCount` drop by one, with no other file edited.
- Every fact matches your CV. You read the six content files and confirm them line by line — this is the moment to catch anything wrong.

The visual and motion vocabulary, before any section exists

Building sections first and extracting a system later is how sites end up with fourteen slightly different greys. The system comes first, and every section afterwards is an application of it.

DECISION GATE AT THE START

Task 2.1 is a review with you: I present two or three complete visual directions — palette, type pairing, and a hero mock for each — and you pick one. Nothing else in this phase starts until you have. This is the cheapest possible moment to change direction.

TASKS

- 2.1 **Direction review with you.** Palette, display and body typefaces, hero composition. You choose one.
- 2.2 Write the token layer in `styles/globals.css` using Tailwind v4 `@theme` — colour, spacing, radii, type scale, shadows, easing curves, durations, both themes.
- 2.3 Self-host fonts: subset to the characters actually used, `@font-face` with `font-display: swap`, preload the display face. No external font request — that would be a third-party dependency and a privacy leak.
- 2.4 Write `config/motion.ts` — the single home for every timing value in the project.
- 2.5 Build hooks: `useReducedMotion`, `useInView`, `useScrollProgress`, `usePointer`.
- 2.6 Build `chrome/Nav` — generated entirely from the section registry, with scroll-spy and a sliding active indicator.
- 2.7 Build `chrome/ThemeToggle` using the native View Transitions API, with a graceful fallback.
- 2.8 Build `chrome/ScrollProgress` and `chrome/Footer` (footer links also from the registry).
- 2.9 Build UI primitives: `Button` (with magnetic variant), `Chip`, `Reveal`, `SectionShell`. None of them contain content.

2.10 Accessibility pass: skip link, landmarks, focus-visible rings on the token system, tab order.

CONTRACT – CONFIG/MOTION.TS

```
// L3. Every timing value in the project lives here.
// Tuning the feel of the whole site means editing this one file.

export const duration = {
  instant: 0.15, fast: 0.28, base: 0.45, slow: 0.7, ambient: 1.2,
} as const

export const ease = {
  out: [0.16, 1, 0.30, 1],
  inOut: [0.65, 0, 0.35, 1],
  spring: { type: 'spring', stiffness: 260, damping: 28 },
} as const

export const stagger = { tight: 0.04, base: 0.06, loose: 0.09 } as const

export const reveal = { distance: 24, threshold: 0.25, once: true } as const

export const brain = {
  rotationSpeed: 0.0025, // rad/frame → one turn in ~42s
  pointCount: { desktop: 220, mobile: 120 },
  edgeCap: 350,
  pulseInterval: 800, // ms between signal pulses
  parallaxDeg: 6,
  dprCap: 2,
} as const

export const typewriter = { charDelay: 18, thinkingPause: 300 } as const
```

CONTRACT – NAV IS GENERATED, NEVER WRITTEN

```
// Nav.tsx – the entire navigation, derived. Removing a section from
// content/sections.ts removes it from the nav, the page, the scroll-spy,
// and the footer simultaneously. There is no second list to update.

const items = sections
  .filter(s => s.enabled)
  .sort((a, b) => a.order - b.order)

// There is no hard-coded array of links anywhere in this file.
```

EXIT GATE – PHASE 2

- Setting `enabled: false` on a section removes it from nav, footer, and page in one edit.
- Both light and dark themes pass WCAG AA contrast on every text pair.
- Enabling “reduce motion” in the OS makes the shell completely still — not faster, **still** — and nothing is missing or broken.
- The entire site is navigable by keyboard alone, with a visible focus ring at every stop.
- Lighthouse accessibility on the shell: 100.
- Network tab shows **zero** third-party requests.

03 The centrepiece

HERO & 1.5
NEURAL DAYS
BRAIN

The first three seconds decide whether the rest gets read. This phase is the rotating neural brain and the entrance choreography around it.

TASKS

- 3.1 `visuals/brain/geometry.ts` — deterministic point generation from a seeded PRNG, sampled across a brain-shaped volume (two lobes plus stem). Same seed, same brain, every load.
- 3.2 `visuals/brain/project.ts` — Y-axis rotation, perspective projection, depth sorting.
- 3.3 `visuals/brain/render.ts` — the draw loop: points sized and faded by depth, synapse edges between near neighbours, glow from a pre-rendered sprite.
- 3.4 `visuals/brain/pulse.ts` — a signal travelling along a random edge on an interval. This is the detail that makes it read as neural rather than decorative.
- 3.5 `visuals/BaseRing.ts` — the glowing platform from your reference, with a slow rotating gradient sweep.
- 3.6 `components/visuals/NeuralBrain.tsx` — canvas host: DPR handling, resize observer, IntersectionObserver gating, tab-blur pause, reduced-motion static frame, theme-aware recolouring.
- 3.7 `sections/Hero` — headline with per-word stagger, tagline, two actions, derived stat row, assistant teaser line, scroll cue.

- 3.8 Orchestrate the page-load sequence as one timeline, not eight independent animations.
-
- 3.9 Performance pass on a real mid-range phone. Profile, then tune particle counts in `motion.ts`.
-

CONTRACT – CANVAS READS COLOUR FROM TOKENS

```
// Canvas cannot use CSS variables directly. Rather than hard-coding
// colours (which would break R2 and the theme toggle), read the computed
// token values from the host element and re-read whenever the theme changes.

function readPalette(host: HTMLElement) {
  const s = getComputedStyle(host)
  return {
    point: s.getPropertyValue('--brain-point').trim(),
    edge: s.getPropertyValue('--brain-edge').trim(),
    pulse: s.getPropertyValue('--brain-pulse').trim(),
  }
}

// Re-read on theme change so the brain recolours without a reload.
// No colour literal appears anywhere in the visuals directory.
```

CONTRACT – THE RENDER LOOP MUST BE INTERRUPTIBLE

```
// Three independent reasons to stop drawing. All of them must work,
// or the canvas silently burns battery on a backgrounded tab.

if (prefersReducedMotion) { drawStaticFrame(); return } // never animates
if (!isInViewport)         { pause() }                 // IntersectionObserver
if (document.hidden)       { pause() }                 // visibilitychange
```

EXIT GATE – PHASE 3

- Sustained **60 fps on a mid-range Android phone**, measured — not on your laptop.
- The brain completes one rotation in roughly 42 seconds. Slow and ambient, not spinning.
- Scrolling past the hero stops the render loop entirely (verify in the performance profiler).
- Switching to another browser tab stops it too.
- Reduced motion renders one clean static frame — the hero still looks finished, not empty.
- Toggling the theme recolours the brain without a page reload.
- The stat row shows numbers derived from content, matching your CV exactly.

04 JOURNEY, PROJECTS, CONTACT

The three remaining content sections

At the end of this phase you have a complete portfolio you could send to a recruiter, whether or not the assistant ever gets built.

TASKS – 4A JOURNEY

- 4.1 Narrative block: your story in two short paragraphs, composed from profile content.
- 4.2 Timeline: SVG path that draws itself as you scroll, with both internships as nodes. Real dates from content, formatted at render.
- 4.3 Skill clusters with evidence links (or bars, per your decision), staggered on entry.
- 4.4 Education and achievements, understated — supporting evidence, not headline material.

TASKS – 4B PROJECTS

- 4.5 Project composition: problem → approach → outcome → stack → links. Vertical, one at a time, generous spacing. Not a grid of cards.
- 4.6 Entry choreography: clip-path wipe, staggered content, pointer-tracking light on hover.
- 4.7 Stack chips rendered from `project.stack`, deduplicated and consistently styled.
- 4.8 Graceful handling when a project has no live or repo link — the layout must not leave a hole.

TASKS – 4C CONTACT

- 4.9 Availability statement and email, from profile content.
 - 4.10 `visuals/wave.ts` — the signal wave from your reference, amplifying as the section enters view.
 - 4.11 Social links from `profile.links`, with icons mapped by `kind` — adding a link is a content edit, not a code edit.
-

4.12 Full responsive pass across all sections: 360 px, 768 px, 1024 px, 1440 px, 1920 px.

CHOREOGRAPHY DISCIPLINE

Each section enters *differently*: the timeline draws, projects wipe, clusters stagger, the wave amplifies. The same fade-up applied five times is the single clearest tell of a generated site, and it is what most portfolio templates do.

EXIT GATE — PHASE 4

- **The portfolio is complete and shippable.** Every section renders real CV data.
- Lighthouse on mobile: Performance ≥ 95 , Accessibility 100, Best Practices 100, SEO 100.
- The literal scan finds **zero** content strings in `components/`.
- Adding a fifth project to `projects.ts` makes it appear in the Projects section, updates the hero counter, and updates the technology count — with no other file touched.
- No horizontal scroll at any width from 320 px upward.
- Cumulative Layout Shift below 0.05 with all animations running.

05 PORTFOLIO ASSISTANT DAYS **The zero-cost answer engine**

A real retrieval system running entirely in the visitor's browser. No API, no key, no server, no bill — and it cannot invent a fact, because every sentence is composed from validated content.

TASKS

- 5.1 `assistant/passages.ts` — turn the content layer into ~35 retrievable passages: one per project, per role, per skill cluster, per achievement, plus profile and contact.
- 5.2 `assistant/tokenize.ts` — lowercase, strip punctuation, remove stopwords, light suffix stemming, then expand through the vocabulary map.
- 5.3 `scripts/build-index.ts` — build step producing a static BM25 index (~25 KB), written to `lib/assistant/index.generated.json` and committed.

- 5.4 `assistant/retrieve.ts` — BM25 scoring with parameters from config, returning ranked hits with scores.

- 5.5 `assistant/intents.ts` — ordered matchers: project-detail, skill-check, experience, education, achievements, availability, contact, overview.

- 5.6 `assistant/compose.ts` — one composer per intent, each returning structured answer blocks built from content.

- 5.7 `assistant/suggest.ts` — suggested questions generated from content, filtered by the section currently in view.

- 5.8 `assistant/fixtures.test.ts` — ~40 question→expected-topic pairs including misspellings, unknown topics, and deliberate misses.

- 5.9 `sections/Assistant` — the conversational UI: input, thinking dots, typewriter reveal, rich answer blocks, suggestion chips.

- 5.10 Vocabulary tuning pass — run realistic questions, add the aliases that were missing. This is data editing, not coding.

CONTRACT – THE ANSWER PIPELINE

```
// Every type here is structural. Answers are data, rendered by components.
// No composer ever returns a formatted HTML string.

type PassageKind =
  | 'profile' | 'experience' | 'project'
  | 'skill'   | 'achievement' | 'education' | 'contact'

type Passage = {
  id: string
  kind: PassageKind
  title: string
  text: string           // searchable body
  tags: string[]         // entities: technologies, org names
  sectionId: string      // where to jump on click
}

type AnswerBlock =
  | { type: 'text';      value: string }
  | { type: 'chips';     values: string[] }
  | { type: 'project';   id: string }
  | { type: 'timeline';  ids: string[] }
  | { type: 'link';      label: string; href: string }

type Answer = {
  blocks: AnswerBlock[]
  confidence: number // drives the fallback path
  sources: string[]  // passage ids – provable grounding
}

type Intent = {
  id: string
  test: (query: NormalizedQuery) => boolean
  compose: (query: NormalizedQuery, hits: Hit[]) => Answer
}
```

CONTRACT – ANSWERS ARE COMPOSED, NEVER CANNED

```
// Every sentence is slot-filled from validated data. Change the CV,
// and the answer changes with it. There is no answers.json to maintain.

const composeProjectDetail = (project: Project): Answer => ({
  blocks: [
    { type: 'text',    value: `${project.name} – ${project.problem}` },
    { type: 'text',    value: project.approach },
    { type: 'chips',   values: project.stack },
    { type: 'text',    value: project.outcome },
    { type: 'project', id: project.id },
    ...project.links.map(l => ({ type: 'link' as const, ...l })),
  ],
  confidence: 1,
  sources: [`project:${project.id}`],
})
```

CONTRACT – THE HONEST FALLBACK

```
// The single most important behaviour. When it does not know,  
// it says so and offers something useful instead of guessing.  
  
if (topScore < threshold && !matchedIntent) {  
  return {  
    blocks: [  
      { type: 'text', value: notFoundMessage(query.raw) },  
      { type: 'chips', values: suggestTopics() }, // generated from content  
    ],  
    confidence: 0,  
    sources: [],  
  }  
}
```

EXIT GATE – PHASE 5

- All ~40 fixtures pass in CI.
- “Tell me about the API monitoring platform” returns its real stack and outcome.
- “Do you know Docker?” answers yes and names the projects that prove it.
- “Have you used Kubernetes?” answers honestly that you have not, and names the closest things you do use.
- “What is your favourite film?” deflects gracefully with three useful suggestions.
- **Disconnect the network entirely and it still works.** This is the proof that nothing is metered.
- Answers render in under 50 ms; any perceived delay is the deliberate pause, not the engine.
- Adding a fifth project makes the assistant able to answer questions about it with **no code change**.

06 SHIP 0.5 DAY Public, findable, and shareable

Everything that only matters once a stranger visits.

TASKS

-
- | | |
|-----|--|
| 6.1 | Metadata generated from content — title, description, keywords, canonical URL. |
|-----|--|
-
- | | |
|-----|---|
| 6.2 | <code>opengraph-image.tsx</code> — social preview card rendered from your name, role, and derived stats. No image file to keep in sync. |
|-----|---|
-

- 6.3 `sitemap.ts` and `robots.ts` , both derived from the section registry.
- 6.4 JSON-LD `Person` schema from profile content — helps you surface in search for your own name.
- 6.5 Favicon and app icon set.
- 6.6 Analytics behind the feature flag, shipped off. Turn on only if you want it.
- 6.7 Domain: connect a custom one, or confirm the free subdomain. Per your decision.
- 6.8 `README.md` — a real runbook: how to add a project, edit a fact, tune the assistant vocabulary, adjust motion, deploy.
- 6.9 Cross-browser and device matrix: Chrome, Safari, Firefox, Edge · Android and iOS.
- 6.10 Final Lighthouse run and a manual accessibility walkthrough with keyboard only.

EXIT GATE — PHASE 6

- Pasting the URL into LinkedIn and WhatsApp shows a correct preview card.
- Searching your name plus “portfolio” finds the site once indexed.
- A stranger can clone the repo and run it with **no configuration and no accounts**.
- The README lets *you*, six months from now, add a project without re-reading any source file.
- Every gate in Blueprint §10 is met and recorded.

07 BEYOND ONBOARDING V1

How this grows over the next few years

Not scheduled work — a map of what the architecture makes cheap later, so nothing built now blocks anything wanted later. Every item below stays at zero cost.

HORIZON	ADDITION	WHY IT IS CHEAP
0–3 months	Turn the blog flag on; MDX case studies for the strongest two projects	The shell is already built and the section registry already supports it.

HORIZON	ADDITION	WHY IT IS CHEAP
0–3 months	Generate your résumé PDF <i>from</i> the content layer	The data is already typed and validated. The CV stops being a separate document that drifts.
3–12 months	Per-project deep-dive pages	Projects are already addressable by id with a schema that supports long-form fields.
3–12 months	Assistant analytics — which questions get asked, stored locally	Tells you what recruiters actually want to know, so you write the content they are looking for.
Year 2	Publish the answer engine as an open-source package	It is already isolated, pure, and tested. This becomes a portfolio item in its own right.
Year 2–3	Optional LLM upgrade behind the existing flag	The flag and the fallback are built in Phase 5. If a free tier ever becomes dependable, it is a config change — and the zero-cost engine remains the default.
Year 2–3	Extract the content layer into a reusable template	The whole architecture is already content-agnostic. Someone else's data, same site.

THE POINT OF THE ARCHITECTURE

Every row above is cheap for the same reason: content is data, the structure is derived, and nothing is hard-coded. That constraint you set at the start is what buys all of this — a portfolio you update in one file for the next several years, instead of a site you rebuild every time your career moves.

Σ SEQUENCE

The whole plan at a glance

PHASE	OUTPUT	EFFORT	BLOCKED BY
0 · Foundation	Deployed empty shell, rules mechanically enforced	0.5 d	—
1 · Content	Your CV as validated data	0.5 d	Your project links, if any
2 · System	Tokens, motion config, nav, chrome	1.5 d	Your direction pick
3 · Hero	Rotating neural brain, entrance	1.5 d	Brain decision
4 · Sections	Complete shippable portfolio	2 d	Skills decision
5 · Assistant	Zero-cost answer engine	1.5 d	—
6 · Ship	Public and findable	0.5 d	Domain decision

BEFORE PHASE 0 STARTS

Only two of the four open decisions block early phases, and neither blocks Phase 0 or 1 — so we can begin immediately and settle them as we go. What would genuinely help now: your **GitHub and LinkedIn URLs**, and any **live or repository links** for the four projects. A “view source” link is worth more to a reviewer than any description on the page.

Say “start Phase 0” and I will begin. Nothing is written until then.